Machine Learning Environment for the Isabelle Proof Assistant

Zijing Li

Master of Science
School of Informatics
University of Edinburgh
2025

Abstract

The combination of machine learning and interactive theorem proving (ITP) to achieve automated mathematical theorem proving has become a cutting-edge field of great research value and has been implemented in various proof assistant environments. However, as a mature and powerful proof assistant, Isabelle lags behind in terms of its ecosystem for machine learning interaction environments and the latest research projects from the Lean community. This project aims to maintain and enhance the existing IsabelleGym environment in multiple aspects to build a more general and scalable machine learning pipeline.

The project successfully implemented a standardised, unified agent interface that supports easy integration of search algorithms such as Monte Carlo Tree Search (MCTS) and reinforcement learning (RL). It also provides structured, convenient data extraction capabilities for machine learning training, improving the modularity of the project structure. The back-end architecture of the system has also been optimised in depth, introducing multi-session management mechanisms and LRU (Least Recently Used) caching strategies to improve the interaction and stability of the system. This project also improved the original project code, resolved version incompatibilities and format inconsistencies, improved the deployment process, and provided comprehensive development documentation, effectively reducing the learning cost for researchers. After multi-dimensional experimental evaluations, this project showed significant improvement over the original IsabelleGym, providing a foundation for the Isabelle community to catch up and surpass others in the field of automated theorem proving (ATP).

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Zijing Li*)

Acknowledgements

I am deeply grateful to my supervisor, Wenda Li, for his guidance and support throughout this project. He is a dedicated mentor who provides me with valuable academic advice that has allowed me to advance my research smoothly. I would also like to express my heartfelt thanks to my parents for their unwavering support and encouragement during the most challenging times for me.

Table of Contents

Chapter 1

Introduction

Formal theorem proving is one of the most important research directions in computer science and mathematics today. With the large-scale rise of artificial intelligence and machine learning around the world, the use of machine learning methods and proof assistants to achieve automated theorem proving has received increasing attention [1]. Proof assistants such as Lean, Isabelle, and Coq have all developed interactive machine learning environments for formal theorem proving in recent years [19]. In particular, the combination of large language models (LLMs) ensures the correctness of each step of reasoning and avoids the 'hallucination' problem that often occurs in AI models [18].

Among these, DeepMind developed the self-training AlphaProof system, which uses pre-trained language models and the AlphaZero reinforcement learning algorithm to prove formal problems in the Lean environment, solving complex problems at the level of the International Mathematical Olympiad [4]. These successful implementations of formal proof projects that use reinforcement learning and LLM have made the integration of automated theorem proving and artificial intelligence more promising. This also highlights that the current machine learning pipelines still have significant room for improvement before they can achieve the ideal development environment for AI agents and human users.

1.1 Motivations

In the field of mathematical proof, automated theoretical proof (ATP) has been highly sought after by academic researchers since the emergence of machine learning concepts. However, due to the complexity of proof states and environments, human intervention

and interaction are necessary in the proof process, giving rise to interactive theoretical proof (ITP). ITP has been developed to a considerable extent in many different proof assistants, and a standardised, stable interactive environment is an essential foundation for machine learning and proof assistant integration. Such environments are typically designed in a gym-like format to provide a benchmark platform that allows developers to test and compare different algorithms more easily, thus freeing up more resources to focus on essential research directions [22]. Gym environments have been successfully applied in proof assistants such as Lean and Coq, and numerous valuable research platforms have been built on top of them. In contrast, there is currently no comprehensive machine learning interactive environment for Isabelle, and it lacks unified development standards, resulting in a significant gap between it and the latest research results of theorem-proving assistants such as Lean 4.

To address this issue, the newly implemented IsabelleGym, developed by Milan Tom from the University of Cambridge, which is also the research foundation of this project, provides a successful pipeline connecting Isabelle, Scala and Python, and builds a feature-rich gym environment based on the PIDE architecture [21]. This research is an unpublished bachelor's thesis and can be found in the supplementary material. However, despite the significant breakthroughs and progress made in this foundational work, its implementation of algorithms such as reinforcement learning and MCTS is mostly scenario-specific, lacking broad applicability in terms of machine learning algorithms and architecture choices, which limits the depth of this project in AI-driven automated theorem proving.

Furthermore, in the current context of further cooperation between large language models and interactive theorem provers, IsabelleGym has also failed to provide a sufficiently robust research foundation for the extraction of high-quality, structured training data. Therefore, to bridge the gap between Isabelle and other theorem-proving assistants, this project will reconstruct and optimise the IsabelleGym project by learning from outstanding machine learning projects in other theorem-proving assistant communities in the academic field, catch up with the competition, and provide a reliable platform foundation for the training of large language models in the future.

1.2 Research Objective

Based on the above motivations, this project has set the following main objectives:

- **Construct a generalised intelligent agent framework:** Based on the existing IsabelleGym environment, design and implement a more generalised and robust reinforcement learning interaction framework. This framework will provide reusable high-level interfaces (such as MCTS) to provide standardised support for integrating diverse RL agents and LLMs.
- **Enhance data collection and analysis capabilities for the proof process:** Deeply optimise the system's data extraction capabilities to capture fine-grained information during the proof process, including the evolution of proof states, executed tactics, sub-goal states after tactic application, and the final proof result. This structured data will provide valuable datasets for training supervised learning models and offline reinforcement learning research.
- **Rework and optimise the backend of the IsabelleGym:** Systematically rework and optimise the backend architecture of IsabelleGym to significantly improve the platform's stability, human-computer interaction performance, and overall usability, ensuring the smooth progress of research work.
- **Improve developer experience and project maintenance:** This project is committed to lowering the learning cost and deployment complexity of IsabelleGym, enhancing its user-friendliness for researchers and developers. Additionally, it will systematically address compatibility issues and imperfections in existing projects to improve code quality and long-term maintainability.

1.3 Structure

This paper is primarily divided into five sections.

- **Introduction:** Introduction to the research background and contributions.
- **Preparation:** Discussion of related work, theoretical basis of the experiment, and determination of requirements.
- **Implementation:** Detailed description of the system design and implementation.

- **Evaluation:** Evaluation and verification of the experiment in terms of usability and other aspects.
- **Conclusion:** Summary of research results and shortcomings, as well as prospects for future development.

Chapter 2

Preparation

This chapter provides the necessary background knowledge for subsequent system design and implementation. It introduces the core concepts of the Isabelle proof assistant, reviews the latest research results in other proof assistant communities, especially representative projects in the Lean community, and explains other methods involved in this project, such as machine learning and Monte Carlo Tree Search. Finally, it clarifies the specific requirements of this project to lay the foundation and prepare for subsequent project implementation and other work.

2.1 Isabelle

Isabelle is an interactive theorem prover [14], which is itself an LCF-style proof system developed using the Standard ML language. Its core feature is a very small but reliable logic kernel, through which all complex proof rules and automation tools are ultimately verified to ensure the reliability of the system [13]. Isabelle/Pure is the original logic framework of Isabelle, providing a language and rules for encoding object logics. The most widely used of these is Isabelle/HOL (Higher-Order-Logic), which offers a rich theoretical library and proof tools [12], and serves as the core foundation of our project.

2.1.1 Isar

Isabelle supports both declarative and procedural styles of proof writing. Procedural style is the traditional proof language used in earlier versions of Isabelle, consisting of a series of proof tactics. However, it is no longer recommended due to its difficulty in reading. Intelligible semi-automated reasoning (Isar) [27] is Isabelle's declarative

proof language, which is more readable and maintainable, easier for professionals to understand [28], and more convenient for development in our machine learning environment.

2.1.2 PIDE Architecture

Isabelle's user interaction experience is primarily based on the Prover IDE architecture [25], which is an asynchronous, document-oriented interaction model that replaces the traditional REPL (read-eval-print-loop) [24]. Users can edit the entire theoretical document in front-end IDEs such as JEdit, while Isabelle Prover continuously processes and checks the document content in the background. This asynchronous nature aligns with human users' interaction habits.

2.2 Related Work

The application of machine learning to ATP research has been developing for quite some time. Early work primarily focused on using machine learning to guide heuristic search in traditional ATP [5], such as the third-party automation tool Sledgehammer in Isabelle, which translates the current proof goal and related lemmas into first-order logic, then calls external ATP and solvers to attempt to solve the problem in parallel. Finally, the results of successful proofs are returned to Isabelle [3].

Due to the recent rapid rise of deep learning and large language models, recent research has shifted its focus to end-to-end, learning-based proof generation, including lemma selection (selecting the most relevant lemmas for the current proof goal from a vast library of mathematical proofs), strategy prediction (predicting the next proof strategy based on the current proof position), and proof search (exploring the proof tree using search algorithms such as MCTS to complete the entire proof). Many pioneering works in this field have been conducted in Coq and Lean proof assistants [18], which have advanced the development of automated proof systems.

2.2.1 Literature Review

LeanDojo and PantoGraph LeanDojo is a milestone project in the Lean community. It encapsulates Lean into a reliable and stable Gym interactive environment, greatly reducing the proofreading errors that are common in early tools. It is also the first system

to systematically extract premises, which are fine-grained information that cannot be seen directly, including the state before and after each strategy is applied in the proof process, abstract syntax trees, and other lemma information.

LeanDojo also builds a retrieval-augmented language model called ReProver based on this data, which can retrieve corresponding premises from the matlib library and generate the next proof step, achieving quite impressive test results [30].

PantoGraph is a machine-to-machine interaction interface specifically designed for Lean 4. To address the limitations of LSP interfaces designed for human users in automating intelligent agent proofs, it introduces a comprehensive machine-oriented API with the following key features:

1. Support for advanced reasoning: PantoGraph can handle complex proof states, such as subgoal dependencies caused by meta-variable coupling, and supports advanced reasoning strategies like 'have', enabling the construction of more complex proofs.

2. Data extraction and interaction: Supports extracting proofs as programs (S-expressions) and recovering proofs from incomplete proof drafts with 'sorry' placeholders. This represents an innovative breakthrough for DSP (Draft-Sketch-Prove), where LLMs generate proof sketches that are then completed by automated tools.

PantoGraph provides a solid-oriented interface from design concepts to data extraction capabilities, inspiring the redesign of the interface in this project.

CoqGym and Tactician CoqGym created a large-scale dataset of 71,000 manually written proofs for supervised learning [29]. By extracting intermediate states and expert decisions from the proof process, CoqGym trained early deep learning models (ASTactic) for strategy prediction, representing strategies as abstract syntax trees. Building on this, Tactician [2], as a Coq plugin, runs directly in the user's development environment, learning in real time from proofs written by the user. Based on the knowledge it has learned, Tactician can provide users with predictions for the next strategy or attempt to automatically complete the proof. Tactician itself is also an extensible machine learning platform that allows different prediction models to be inserted.

Kimina-Prover Kimina-Prover is a large language model specifically designed for the Lean 4 proof assistant, achieving state-of-the-art results on the miniF2F benchmark [23]. It pioneers an agent paradigm driven by reasoning, guiding the LLM to perform long-chain reasoning through large-scale reinforcement learning, rather than relying on external search algorithms. This training paradigm requires extensive interaction. To address this, the backend verification server Numina Lean Server implements a critical performance optimisation: a caching mechanism based on the Least Recently Used (LRU) strategy. By reusing preloaded environments based on import headers, it significantly reduces redundant initialisation overhead, achieving a 10-fold improvement in verification throughput.

2.2.2 PISA and IsabelleGym

PISA (Portal to Isabelle) was an early attempt by the Isabelle community to connect machine learning and proof assistants. It provided feasibility verification for building interactive environments in Isabelle and was used to extract a large-scale proof corpus from Isabelle, providing data support for the development of language models such as LISA [8]. However, PISA relies on the sacla-isabelle library, which is not officially supported, resulting in additional performance overhead and making long-term maintenance and compatibility challenging. Additionally, the proof states extracted by PISA are incomplete, containing only user-visible proof goals and ignoring the full proof context, which does not satisfy the Markov property required for reinforcement learning [21].

The predecessor of this paper's work, the foundational implementation project named IsabelleGym, built a modern and feature-rich machine learning interaction environment for Isabelle/HOL, addressing the limitations of previous tools like PISA. It implemented a new interaction workflow using the native official Isabelle/Scala and PIDE architecture, while maintaining parallelism during asynchronous interaction. It encapsulates the inherent interaction model into a REPL interface that meets the requirements of reinforcement learning agents [21], and builds a feature-rich Gym environment on top of this workflow. Although this is a significant achievement, providing a functional and high-performance environment, its primary validation is focused on specific examples within a narrow domain. In other problem domains, the machine learning methods used

cannot be directly applied. This is the core problem this project aims to address: generalising the agent interface designed for specific problems to enhance system usability, while further optimising backend performance to create a more general-purpose Isabelle machine learning environment research platform.

2.3 Techniques

2.3.1 Monte Carlo Tree Search

Monte Carlo Tree Search is a heuristic search algorithm mainly used for large-scale decision-making problems. It iteratively constructs a search tree to find the most promising path within limited time resources, thereby providing an approximate optimal strategy. MCTS has achieved great success in fields with huge search spaces, such as Go [31].

Each iteration of MCTS consists of four steps: Selection, Expansion, Simulation, and Backpropagation. Throughout the iteration loop, the search tree continues to expand until it reaches a predefined upper bound.

Selection: Starting from the root node, recursively select child nodes according to a tree policy and traverse downward until reaching an unexplored leaf node (i.e., a node with no visited or evaluated child nodes). Kocsis and Szepesvri proposed the UCT (Upper Confidence Bound Apply to Tree) algorithm in 2006 [9], combining the UCB formula and the MCTS method to maximise total utility within a limited number of trials.

$$UCT(v) = \frac{Q(v)}{N(v)} + C\sqrt{\frac{\ln N(parent(v))}{N(v)}}$$

In this formula, $Q(v)$ is the total reward accumulated or number of wins after selecting action v in subsequent simulations, $N(v)$ is the number of times node v has been visited, C is the exploration parameter that controls the balance between exploration and exploitation, and $parent(v)$ denotes the parent node of v.

This method balances exploration of known paths and exploitation of unknown paths, becoming the gold standard for the selection phase of MCTS [7]. It has been adopted by many modern MCTS implementations and has been extended to many variants,

including the subsequent representative AlphaGo and AlphaZero projects.

Expansion: If the selected leaf node is not a termination state, one or more new child nodes are created, representing new states that will result from untried actions. Then, according to the set algorithm, one of these child nodes will be selected to enter the next stage of simulation. The main contribution of this stage is to expand the boundaries of the search tree, allowing more unexplored states to enter the evaluation range.

Simulation: Starting from the node selected in the previous stage, the algorithm executes a complete simulation until a termination state is reached. In the earliest classic MCTS, this step was completed using uniform random or heuristic strategies.

Backpropagation: The results of the simulation, such as wins and losses, are propagated back to the root node along the path of the selection stage, while updating the cumulative rewards and visit counts of all nodes along the path.

2.3.1.1 MCTS in AlphaZero

AlphaZero innovatively combines value networks, policy networks, and the MCTS framework, using value networks to replace the N rollouts in the simulation phase to provide more accurate positional evaluation and improve computational efficiency [17].

In AlphaZero's MCTS cycle framework, the UCT followed by selection is updated to PUCT (Polynomial Upper Confidence Trees). The PUCT value of action a selected from parent node s is calculated as follows:

$$PUCT(s,a) = Q(s,a) + U(s,a)$$

where *Q(s,a)* is the average return from all simulations after taking action a, representing the experience value obtained through search, and *U(s,a)* is defined as:

$$U(s,a) = c_{puct} P(s,a) \sqrt{\frac{\sum_b N(s,b)}{1+N(s,a)}}$$

where *P(s, a)* refers to the prior probability of action a, provided by the policy network. This is the core improvement of PUCT over UCT, as it injects the prior knowledge from the neural network into each decision step of the search. *N(s, a)* represents the number

of times the edge (s, a) has been visited, and *c_puct* is a hyperparameter that controls the exploration-exploitation balance, similar to C in UCT.

The main difference between PUCT and UCT lies in the bias toward actions that have not been sufficiently explored. By weighting the action selection probability *P(s, a)* with the strategy prior probability, the priority of visits is adjusted, enabling selective exploration. This is crucial for integrating MCTS and reinforcement learning in our project.

2.3.1.2 MCTS in IsabelleGym

AlphaZero's neural network self-play loop requires a lot of computational resources, making it difficult to reproduce. IsabelleGym adopts a decoupled strategy, first using a TD-learning (temporal difference learning) algorithm to independently train a reinforcement learning agent [16], and then encapsulating this agent in an AlphaZero-style MCTS during actual proof search to achieve the goal of 'train first, search later' [21].

This training method greatly reduces the dependence on computational resources and avoids expensive, large-scale self-play training, making it more suitable for IsabelleGym academic research. Therefore, the MCTS implementation in this project is also based on this training method.

2.4 Project Management

2.4.1 Code Management

To prevent source code confusion, version mix-ups, and loss of work progress, I use GitHub to submit code regularly. Each code submission is 100% guaranteed to be an error-free update. Whenever issues or anomalies are discovered, I can quickly revert to the previous version, thereby resolving issues such as work loss and version inconsistencies.

2.4.2 Version Management

Every time I modify, update, or expand the original logic and code, I record the work log in a Markdown document and tag it with a time-file-code location data string to ensure

that all updates can be tracked and found. In addition, I also use agile development, minimise deliverables, and modular management to provide a solid foundation for project implementation.

Chapter 3

Implementation

This chapter provides a detailed overview of the design and implementation of the core components of IsabelleGym 2.0, all of which are built upon the foundational implementation described in Section 2.2.2. The entire implementation process is structured around a three-tier architecture comprising Isabelle, Scala, and Python, introducing a modular and extensible framework to apply advanced search algorithms to theorem proving in Isabelle/HOL. This section will focus on the implementation of the agent interface and LRU backend cache management system, as well as the maintenance and extension of the gym environment, enabling readers to better understand the hierarchy and value of the implemented work. This chapter also uses visual diagrams, such as system architecture diagrams, to illustrate the implementation work more intuitively.

3.1 Agent Interface

The PantoGraph project proposes a highly generalised and core cognitive concept: interfaces designed for human users are fundamentally unsuitable for machine agents. The standard way of interacting with Lean 4 proof assistants is LSP (language server protocol), and this interaction-oriented design philosophy poses challenges for automated proving. To address this, PantoGraph created a machine-oriented REPL and API, exposing the entire proving process as a state system that can be manipulated by programs. By applying tactics to specific goals, obtaining explicit information such as relevant sub-goals, and managing the proof tree using functions [20], Pantograph can achieve agents with powerful search capabilities, which is precisely because it is agent-centric. This agent provides standardised interfaces, keeping the main search

loop generic while allowing the free selection of different implementations of agents to provide specific search strategies for actual scenarios.

In IsabelleGym, the same ideas and design style exist. Although Isabelle and Lean proof assistants are different, they are similar in their structure for moving towards automated proof search. This is also due to the modular architecture used by IsabelleGym itself, which makes the implementation of agent interfaces more convenient and powerful.

- Modularity: The Gym environment attempts to separate agents from the proof environment. The implementation of the agent interface allows subsequent researchers to test and develop various proof search algorithms on the platform provided by the agent without having to consider the underlying complex communication logic with Isabelle.
- Generality: Using any specific proof search strategy, such as reinforcement learning, various MCTS methods, or LLM-based provers, is not universally applicable in any single problem scenario. A standardised agent interface not only provides code reuse optimisation but also serves as a unified implementation foundation for machine learning-based automated proof.

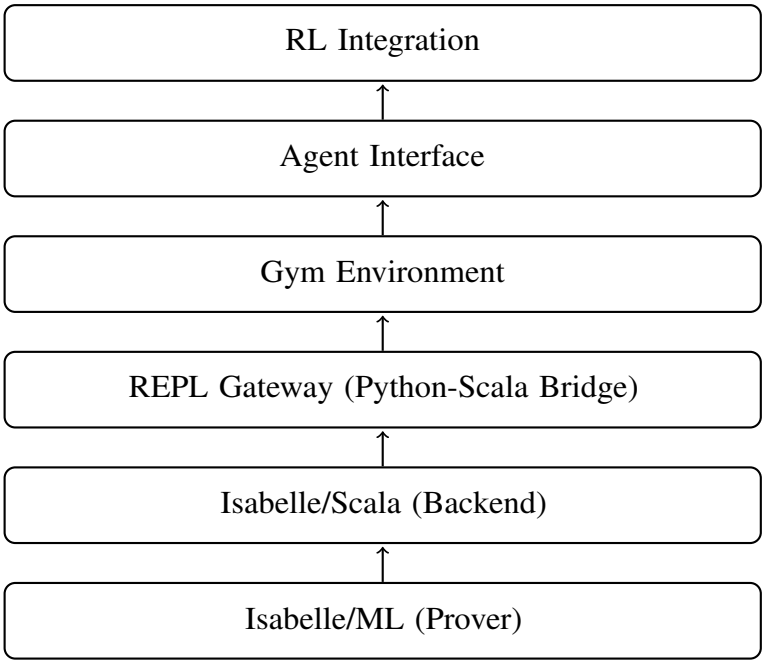

Figure 3.1: Bottom-up system hierarchy diagram

Figure 3.1 shows the bottom-up structure from a system perspective. The design of the agent interface primarily draws inspiration from the interface design in PyPantograph [20], with optimisations and improvements tailored for Isabelle. A multi-layer architecture is employed to separate the complexity of the proof assistant from the upper-level Agent implementation, thereby ensuring robust maintainability and the potential for seamless scalability to various machine learning search algorithms.

3.1.1 Core interface design

Several important abstract class methods are defined in IsabelleAgent.

- *recommend_tactic* generates the next tactic for the specified goal ID.
- *evaluate_priorities* returns a priority list for each subgoal, and search executes the complete proof search logic. All classes integrated from this agent can independently reimplement these methods and implement specific solutions for priority determination for specific problems, greatly satisfying code-level reusability and specific problem requirements.

3.1.2 Search State Management

The ProofContext class manages the complete state information of the proof search, including identifying the highest-priority unsolved goal and selecting it. For instance, if the default priority assigns equal weight to each subgoal, the system will solve them in FIFO-DFS order according to the rules.

- **ProofContext**(data class): The core methods provided by this class implement a multi-subgoal priority scheduling mechanism, synchronising the state as the proof progresses. Table 3.1 shows the main attributes in ProofContext.
- **ProofResult** (data class): Also inspired by the implementation in PantoGraph. It has been extended with the following attributes that better capture performance metrics:
- **guidance_calls**: Measures the dependency on heuristics by recording the number of times guidance was invoked.
- **tactic_recommendations**: A history of recommended tactics, used for training data collection.

Table 3.1: Attributes of the ProofContext data class

Attribute	Function	Update Mechanism
`subgoals`	Stores the current subgoals of the Isabelle proof.	Updated via the `gym.open_subgoals()` method.
`priorities`	Guides the search algorithm in prioritising subgoals.	Calculated by the `evaluate_priorities()` method.
`solved`	Marks subgoals that have been successfully proven.	Works in conjunction with the `is_complete` property.
`attempted_tactics`	Stores tactics that have already been attempted.	Prevents repeating tactics and entering infinite loops.

- **duration**: The total time elapsed, allowing developers to evaluate the algorithm's efficiency.

3.1.2.1 IsabelleAgent

The foundation of all agent implementations. Any class that integrates IsabelleAgent must implement two core abstract class methods:

- recommend_tactic: Generates the next recommended tactic for achieving the goal.
- evaluate_priorities: Outputs a list of priority scores based on the current environment and proof state.

Figure 3.2 illustrates the core search loop process. After obtaining sub-goals and creating the search state, the main search loop is entered. Based on guidance and completion strategies, selection and proof are performed and the entire process is tracked by state monitoring methods. The Agent Interface also provides a SimpleAgent implementation for benchmarking, using the simplest strategy selection and FIFO+DFS with zero priority as an unbiased baseline for integrating RL and other search algorithms.

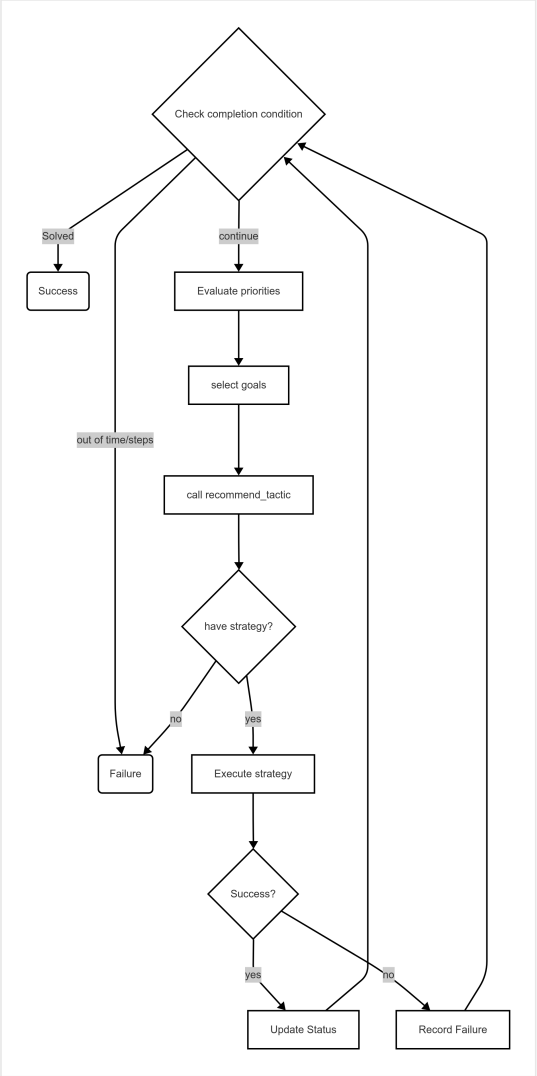

Figure 3.2: Main Search Loop

3.1.3 Architecture

The agent interface is built on the gym layer, which is a standardised implementation interface for all intelligent Agents, and the integration of data extraction and visualisation capabilities. Figure 3.3 shows the architecture of the front-end Agent-Interface layer.

3.1.4 Training data extraction and Visualisation

TrainingData Collector is responsible for standardising and recording the proof process, including the goal state before execution, the goal state after execution, the executed tactic, whether it was successful, the goal ID, all subgoals before and after execution, and the execution time, and generating the corresponding JSON files. Ap-

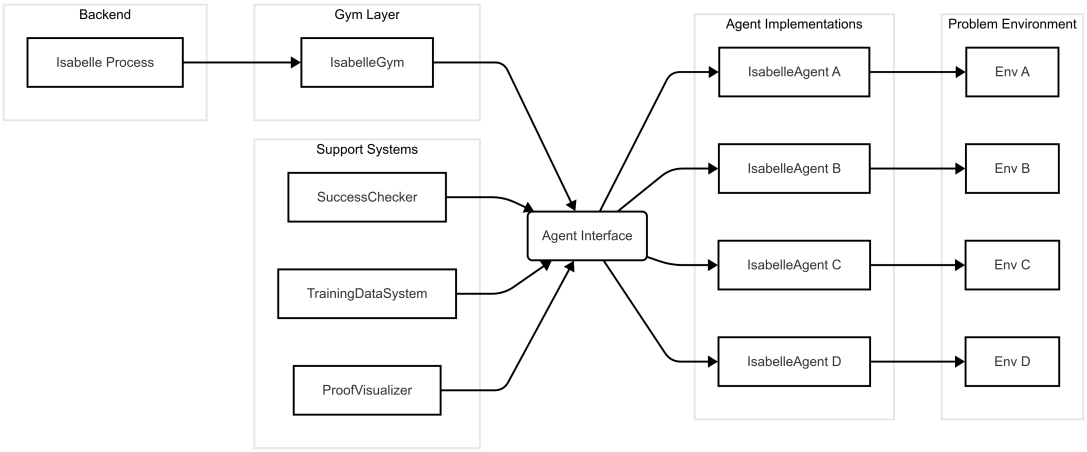

Figure 3.3: Architecture of Front-end IsabelleGym

pendix A shows the standardised data format for machine learning.

The purpose of **TrainingData Collector** is to create datasets for machine learning training by capturing information such as the status of each interaction and obtaining the tuples required for supervised learning and offline reinforcement learning. Depending on the purpose, ProofSession also allows to freely select the export format (.json, .jsonl, csv).

ProofVisualizer also provides a proof state visualiser, offering human developers more comprehensive training, monitoring and logging capabilities for strategy visualisation and performance analysis. A basic collaborative workflow for data extraction and visualisation should follow the format illustrated in the example below:

```
# Create a basic agent
base_agent = BaseAgent("BaseAgent")
# Package data collection
collector = TrainingDataCollector("training_data")
training_agent = TrainingDataAgent(base_agent, collector)
# Integrate visualisation
visualizer = ProofVisualizer()
```

A complete data format is provided in Appendix A to more visually demonstrate the capabilities of data extraction.

Table 3.2: Comparison of Training Data System and Proof Visualizer

Feature	**Training Data System**	**Proof Visualizer**	**Synergistic Advantages**
`Data Collection`	Structured data for ML training	Human-readable execution logs	Dual recording for complementary validation
`Performance Analysis`	Quantitative metrics (time, success rate)	Intuitive progress display	Quantitative and qualitative analysis
`Debugging Support`	Detailed call tracing	Real-time state display	Comprehensive error diagnosis
`Research Value`	Large-scale training datasets	Understanding of proof processes	Supports different research needs

3.1.5 MCTS Integration

The MCTS agent is integrated from the general IsabelleAgent and ProofContext, then expanded and specifically implemented using the MCTS algorithm. Based on the unified interface of IsabelleAgent, I expanded the basic ProofContext according to the characteristics of MCTS to ensure the tree structure required by UCT while maintaining compatibility with the agent interface. All problem proofs or reinforcement learning training that require MCTS capabilities can be implemented in a specific direction on top of this simple but complete basic MCTS agent.

In the early version of IsabelleGym, a SlidingPuzzle example was provided to perform reinforcement learning training in the IsabelleGym environment to demonstrate the capabilities of gym, with independent training, MCTS, and agent implementation. After integrating the agent interface capabilities, I completely replaced the original agent and train execution parts, using a unified interface inherited from IsabelleAgent, adding SlidingPuzzleProofContext, retaining the original MCTS and other strategies, and introducing data extraction and proof visualisation while remaining fully compatible with the outside world, covering complete TD learning and course-based difficulty

increase training. After updating the unified agent interface, the old puzzle environment, MCTS, and TD learning files can be reused without any changes. Any upgrades and new reinforcement learning methods can be directly integrated, and supervised pre-training can be selected without any adjustments to the Agent layer.

Through training, the new SlidingPuzzle based on the Agent Interface maintains the same model performance as the original standalone environment implementation and demonstrates good model performance within the scramble $\leq$10 range. Below is a test demo with a scramble of 10.

```
python -m sliding_puzzle.demo --scramble 10 --max-steps 80

Start board (  10 scrambles): [4, 1, 2, 7, 0, 5, 8, 6, 3]
Step 01 | RIGHT | [4, 1, 2, 7, 5, 0, 8, 6, 3]
Step 02 | DOWN  | [4, 1, 2, 7, 5, 3, 8, 6, 0]
Step 03 | LEFT  | [4, 1, 2, 7, 5, 3, 8, 0, 6]
Step 04 | LEFT  | [4, 1, 2, 7, 5, 3, 0, 8, 6]
Step 05 | UP    | [4, 1, 2, 0, 5, 3, 7, 8, 6]
Step 06 | UP    | [0, 1, 2, 4, 5, 3, 7, 8, 6]
Step 07 | RIGHT | [1, 0, 2, 4, 5, 3, 7, 8, 6]
Step 08 | RIGHT | [1, 2, 0, 4, 5, 3, 7, 8, 6]
Step 09 | DOWN  | [1, 2, 3, 4, 5, 0, 7, 8, 6]
Step 10 | DOWN  | [1, 2, 3, 4, 5, 6, 7, 8, 0]
Solved in 10 steps
```

The general capabilities provided by agent interface can also be successfully integrated into other formalised problem scenarios, such as Tic-Tac-Toe and pass unit tests. An excerpt from the Isabelle theory illustrating the formal proof steps.

```
theory SlidingPuzzleSolution imports
SlidingPuzzle
begin

definition scrambled_board :: board where
"scrambled_board = [Tile 4, Tile 1, Tile 2, Tile 7, Gap, Tile 5,
                                  Tile 8, Tile 6, Tile 3]"

theorem scrambled_solvable: "solves_n_by_n_puzzle 3 scrambled_board"
unfolding scrambled_board_def
unfolding solves_n_by_n_puzzle_def
apply (move 5)
apply (move 3)
```

```
apply (move 6)
apply (move 8)
apply (move 7)
apply (move 4)
apply (move 1)
apply (move 2)
apply (move 3)
apply (move 6)
```

After experimentation, the generic capabilities provided by the Agent Interface can also be accessed properly in other formal problem scenarios, such as Tic-Tac-Toe.

3.2 LRU Cache System

In Isabelle/HOL, a session represents an independent environment composed of a set of related theories built on top of a given parent session, with these theories recursively traced back to the root object logic Pure. Starting a session involves loading a precompiled heap image [26] that represents the persistent state of the parent session, containing all the logic, theorems, and data structures required for proof checking within the session context. Loading this heap requires a certain amount of time and computational resources, and for complex sessions that import larger libraries (such as AFP), the startup process can be more time-consuming [6], which is incompatible with the environmental requirements of reinforcement learning training.

IsabelleGym has successfully build a pipeline from Isabelle to the Python frontend, but there are still many limitations:

1. Each Isabelle environment only supports single-user conversations.
2. Repeated theory loading causes performance overhead and waste, which is not friendly to the training process of some machine learning algorithms.
3. Resources cannot be properly released after a session is abnormally terminated.

Therefore, this project aims to establish a persistent multi-session environment and design a multi-session management and LRU caching system to address the above challenges. This section will explain the LRU algorithm, memory management optimisation, exception handling mechanisms, and the bottom-up integration of the entire system architecture.

3.2.1 Core Implementation

In a reinforcement learning environment, agents need to frequently create and manage Isabelle sessions, and each new session requires loading the complete theoretical files and the Isabelle kernel. Repeatedly creating sessions with the same configuration wastes memory resources [11]. The training process of Kimina-server relies on a dedicated Numina Lean Server backend, which can achieve 100 verifications per second on a large CPU cluster [23]. This high-performance theorem proving backend utilises an LRU session cache mechanism to reuse preloaded environments, assuming that recently used sessions will be reused soon [15]. In this environment, agents are allowed to execute multiple interactions in the same proof context and then move on to other new proof contexts, achieving significant acceleration. This innovative implementation provides an important reference value for the improvement of the backend of this project.

Without compromising any original functionality or compatibility, this system adopts a three-layer architecture and introduces an LRU caching mechanism.

- **Process pool**
  The backend architecture is based on an independent Isabelle process pool running on the operating system. Each process has a running Poly/ML environment and loads specific Isabelle sessions, such as HOL or HOL-Eisbach, into memory via a heap image [26].

- **Session Manager**
  A persistent service responsible for the complete lifecycle management of processes in the pool, including process instantiation, communication reuse, resource cleanup, and cache management.

- **Py4J Gateway**
  This layer provides a simple and abstract interface to the IsabelleGym environment, exposing the required IsabelleGym environment (theoretical files, etc.) while hiding the underlying complexity of the process pool, cache mechanism, and lifecycle management. For AI agents or human users, the system retains the same logic and usage as the original system without enabling multiple sessions and caching.

3.2.1.1 Data Structure

To reuse sessions that have already been initialised and completed theory loading, avoiding redundant overhead, the Session Manager implements an LRU cache mechanism, primarily relying on two synchronised data structures to ensure session safety.

```
//Session cache pool: Theory list -> session queue
private val session_cache: Synchronized[mutable.Map[List[String],
mutable.Queue[Session_Data]]]
//LRU order management: Theory list -> Last access time
private val lru_order: Synchronized[mutable.LinkedHashMap[List[
    String], Long]]
private val cache_stats: Synchronized[mutable.Map[String, Int]]
```

Session cache uses a hash table to map theoretical configurations to a session queue, allowing multiple sessions with identical theoretical configurations to be cached. LRU order list is used to record the last access timestamp for each cache key (theory configuration), and cache stats for the cache information.

3.2.1.2 Workflow

- Obtain session: When repl_session requests a new session, the session manager first checks whether caching is enabled. If caching is enabled:
  - Cache hit: Generate a cache key based on the initial theoretical list. If a corresponding non-empty queue is found in the cache pool, it is marked as a cache hit. At this point, the system retrieves the session data, updates the cache key timestamp, and increments the hits count.
  - Cache miss: If the corresponding theoretical list does not exist in the pool or the queue itself is empty, it is treated as a cache miss. The system creates a new isabelle session and adds it to the corresponding key queue in the cache pool, while recording the new timestamp and incrementing the misses and creates statistics.
- Release session: When a session is reset or stopped, it is not destroyed but returned to the cache pool to await reuse.

3.2.1.3 Eviction Strategy

Whenever a new session is added to the cache, the system checks the total number of sessions in the cache against the maximum cache size. If the limit is exceeded, the LRU eviction mechanism is triggered:

1. Find the cache with the earliest timestamp in the queue.
2. Remove the corresponding session from the corresponding queue.
3. The removed session will be destroyed asynchronously to avoid blocking the current session.
4. Remove the corresponding key from lru_order and update the eviction count.

3.2.1.4 Cache Key Design

The efficiency and correctness of caching largely depend on the design of the keys.

- **Key Content:** Cache keys consist of the initial theory list required when the Isabelle session is started.
- **Consistency Guarantee:** The same list of theories in different orders should be regarded as the same configuration, i.e., they should hit the same cache entry (e.g., ['A', 'B'] and ['B', 'A']). Therefore, when generating keys, the system sorts the lists of theories and reuses only sessions with completely identical configurations, ensuring the normativity and consistency of cache keys.

3.2.2 Memory Management

To prevent cache sessions from growing indefinitely and causing memory exhaustion, the system implements an active memory management mechanism. The Session Manager monitors the JVM's memory usage in real time, including used memory, maximum memory, and available memory.

Memory Protection: Allows configuring a maximum number of entries for the cache of each session. When the cache reaches its capacity limit, the LRU strategy is automatically activated. If the available memory of the JVM falls below the set threshold, new sessions will be rejected to ensure that memory usage remains within a controllable range.

Status Monitoring: Users can query memory reports by calling system interfaces for monitoring and debugging purposes.

3.2.3 Exception Handling and Multi-Session Isolation

The system has been designed for robustness in both the Scala backend and Python layers. The frontend user layer implements multi-session management logic, maintaining a sessions dictionary that allows for the creation, switching, listing, and closing of multiple independent instances. Each instance has an independent session in the backend, achieving process-level isolation.

When a Python instance attempts to use a cached session that has been discarded by the LRU mechanism, it will fail because the isabelle process has been closed. The system provides a mechanism to handle session expiry: before use, it checks the validity of the session, and if it has expired, it automatically recreates a new session.

When closing a session, an exception is raised with the message 'error during session removal: No session.' This is because the current session has already been closed, so it is normal behaviour. If the user does not wish to see this message, the exception notification for session closure can be modified to silent handling, which can be done very simply.

3.2.4 Comparison with vectorised environments

In the original IsabelleGym, a method for vectorised environments was implemented by copying the history tree of existing environments in a single Isabelle session, thereby simulating multiple environments in the same process space. This method supports the management of multiple independent Isabelle environments and parallel execution, and allows switching between vectorised and scalarised modes [21]. Although this method is more efficient for tasks with similar exploration paths, multi-session LRU cache management is still necessary.

3.2.4.1 Differences and linkages

LRU cache multi-session and vectorised environments are both aimed at solving multiple independent proof tasks in parallel within a single IsabelleGym environment to

improve theorem-proving efficiency. However, in terms of specific implementation, there are significant differences and application scenarios between the two.

Table 3.3: Comparison of two implementation methods

Features	**LRU multi-session system**	**Vectorised environment**
Parallel model	**Session-level parallelism**: Each session is an independent Isabelle/Scala/JVM process, providing parallel execution capability and robust stability.	**In-Theory Concurrency**: Within a single Isabelle session, multiple environments are simulated within the same process space by replicating the theory AST.
Resource isolation	**Strong isolation**: Session-level isolation. A crash or error in one session has no effect on other sessions.	**Weak isolation**: All environments share the same Isabelle/ML heap, still in the same process.
Resource overhead	**E**ach session has independent memory usage, and starting a new configuration session (when the cache misses) has a large overhead.	**V**ectorising on an existing session is very fast because it only copies the data structures in memory and does not start a new process.
State management	**Independent state**: Each session has an independent proof state, history, and resources.	**Shared root state**: All vectorised environments are forked from the same theoretical state.

A vectorised environment is more suitable for search algorithms such as MCTS. Starting from a saved proof state, it can quickly simulate multiple different tactics and then quickly roll back, thereby avoiding the memory overhead of multiple complete Isabelle processes.

For LRU cache multi-session systems, process-level isolation ensures stability and robustness, making it particularly suitable for long-running, large-scale parallel machine learning training tasks that use multiple agents to explore completely different proof paths in parallel. For repetitive task configurations, the LRU cache can almost eliminate the startup overhead of Isabelle, enabling rapid response.

3.3 Maintenance

The basic version of IsabelleGym has issues such as version incompatibility. The most serious issue is that many methods in older versions have been abandoned, resulting in serious bugs such as the Scala backend failing to compile and the server failing to start. Therefore, this project has restructured methods such as session opening and closing in the backend Scala files.

By referring to the official documentation and responses from official developers on the community forum, and under the forward compatibility standards of Isabelle 2025, I replaced the previously unavailable session creation methods in the session that could not be referenced, abandoned the previous black-box instructions, and handled threads explicitly. I used synchronised units to retrieve the results of asynchronous operations in a synchronised manner, meeting the standards of concurrent programming and ensuring the safety of thread communication. By using more low-level APIs, manual implementation of the current session lifecycle management is required, making the backend server architecture and state transitions more transparent and achieving deep integration. By directly calling Session_Start.command from Isabelle/server_commands.scala, the context is obtained, and the UUID and session are registered with the server, enabling more direct and robust interaction. For process termination, the implementation has been shifted to the Isabelle server core components.

Additionally, the numerous type syntax errors previously present in the backend were caused by Scala 3's changes in syntax, leading to incompatibility with Isabelle. These issues have now been fully addressed, ensuring that the Isabelle build process will not encounter compilation errors.

3.4 Simplified Deployment

When I first started using IsabelleGym, I encountered difficulties with deployment, especially with installing various specific versions of system software and initialising and using tools, as there were no specific instructions or guidance. This deployment method, which relies on developers manually executing multiple steps, is prone to errors, inefficient, and prone to local deployment differences. Therefore, this project introduces a series of engineering deployment solutions.

Easy installation script: The original complex multi-step manual installation process is encapsulated into a single executable shell script, which automatically detects the local system and checks the prerequisites for the deployment command-line tools, operating system type, Java and Python versions, etc., to avoid installation failures due to environment issues. The entire deployment script is executed strictly according to the dependency order: environment check → download and unzip Isabelle (default 2025 version) → create Python virtual environment and install dependencies → initialise Isabelle components → compile Scala code → install and verify. Users can intuitively see progress feedback and success or failure prompts.

Containerised deployment: The system also supports a containerised deployment solution based on Docker, using Python 3.10 as the base image to ensure a clean, minimal Linux environment. All configuration steps are encapsulated in Dockerfile, and key environment variables such as ISABELLE_HOME are fixed to avoid manual configuration.

These improvements reduce the learning cost for new developers, enhance the onboarding experience and development efficiency, and also facilitate long-term maintenance and CI/CD work for the project.

Chapter 4

Evaluation

This chapter compares the functionalities of this project with those of PantoGraph, Kimina-Prover, and the basic version of IsabelleGym through horizontal and vertical comparisons and comparative experiments to demonstrate the characteristics of the functionalities. The test results of the agent interface in the TicTacToe problem are evaluated, the performance of different agents in problem scenarios is verified, and the performance of the LRU cache is tested to comprehensively present the evaluation results of this project.

4.1 Functionality

4.1.1 API Verification

To test the functionality of the Gym interface, a simple agent is created that can randomly select a valid action from the action space at each step and execute it, without concerning itself with the correctness of the actual operation. After experimentation, the Simple Agent was able to interact with IsabelleGym for hundreds of steps without errors or crashes, all functions ran correctly, and passed unit testing. This proves that the core API functionality is implemented in accordance with the IsabelleGym standard.

4.1.2 Correctness Verification

4.1.2.1 Agent Interface

To verify that the system can truly support the implementation of a complete, real-world task scenario, I connected the agent to multiple real-world environments, such as Tic-

Tac-Toe, finding the greatest common divisor, and Sliding Puzzle.

Taking Tic-Tac-Toe as an example, I first wrote a theoretical document to formalise the problem, defined core functions such as valid position checking, obtaining the board position, valid moves, victory detection, and game state, and provided the prerequisites for interaction between the Isabelle environment and the game scenario. Then, based on the agent interface, I reimplemented agents suitable for the Tic-Tac-Toe problem. I compared agents with pure random strategy selection, an MCTS-based agent, and an agent integrating AlphaZero-style neural networks. Performance analysis of the algorithms represented by each agent was obtained, and the results were intuitively demonstrated through data such as time spent per step, search efficiency (step utilisation rate), and guidance efficiency.

Throughout the entire agent strategy execution process, based on the search loop functionality of the MCTSAgent parent class, it allows for visualising changes in subgoals, node values, and strategy execution from the start of the MCTS search. Finally, information such as total time spent, total number of search steps, proof depth, and the complete proof process is provided. This greatly facilitates algorithm selection and analysis for researchers.

As described in Section MCTS Integration, the Sliding Puzzle example has fully demonstrated that integrating the Agent Interface can completely encapsulate all original functionalities without affecting functional completeness. This proves that the agent interface not only has correct API functionality but can also be used to prove formal problems with clearly defined specifications using intelligent agents.

4.1.2.2 LRU Cache System

The functions of the LRU backend cache system can be divided into several parts, which are verified for correctness through unit tests.

- Multi-session management: The system created multiple sessions on a single server and successfully performed operations such as viewing all sessions, viewing the current session, and switching sessions.
- Cache keys: Multiple sessions with different initial theoretical headers are created on the same server, and the number of cache hits and misses is counted to check

whether sessions with the same theoretical header are successfully reused. The results prove that the cache key has a 100% hit rate (which means successful reuse) for sessions with the same theoretical header.

- LRU eviction mechanism: The server creates four sessions and sets the LRU cache pool capacity to two. Through backend status monitoring, it was found that when the third session was created, the eviction mechanism was triggered, and the first session (i.e., the session that had not been used for the longest time) was removed from the session pool. When the fourth session was created, this process repeated itself, removing the third session. Therefore, the correctness of the eviction mechanism was verified.

4.2 Usability

4.2.1 Scalability and Compatibility

This project is based on the original IsabelleGym implementation and uses a modular architecture to encapsulate the Agent Interface layer on top of the front-end Gym layer, supplementing and extending the functionality of the Gym environment while avoiding changes to the original functional design and logic as much as possible. Regarding the use of reinforcement learning algorithms, developers can choose to implement them internally within the Agent or integrate them externally, forming a plug-and-play training method.

The LRU cache mechanism and memory management implementation also maintain compatibility with the original single-session and vectorised environment. To avoid the impact of multiple sessions and LRU cache management on the normal use of the original functions, the system provides multiple optional API entry points in the Scala and gateway layers, allowing users to choose whether to enable the backend with automatic memory management and LRU cache mechanism.

To better validate the usability of the implemented work, I developed a collaborative framework based on a multi-session architecture and agent interface, and compared it with the original collaborative framework to demonstrate the differences in strategy proof and process.

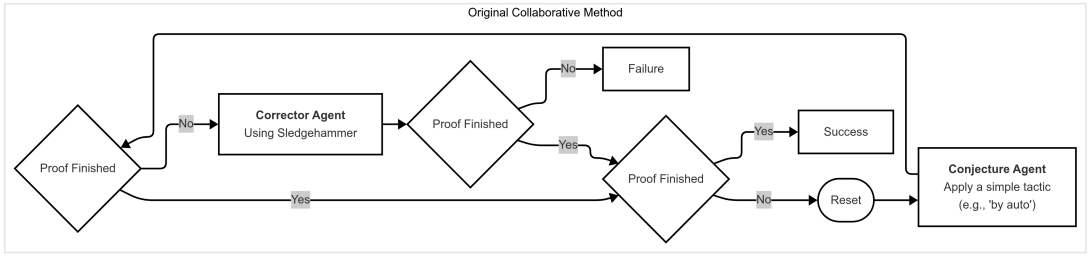

Figure 4.1: Process of original collaborative proving method

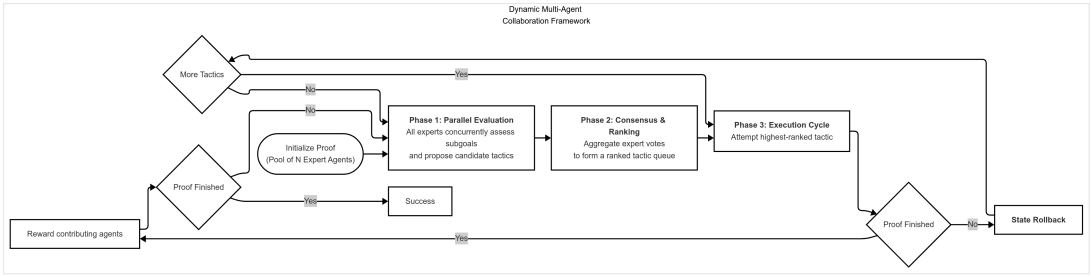

Figure 4.2: Dynamic multi-agent collaborative proving method

The original collaborative proving method used a single-agent sequential execution strategy without a unified strategy selection specification, making state management difficult. The new architecture is based on a unified goal-oriented strategy selection, specialising multiple agents through a three-tier decision-making process: guidance evaluation $\Rightarrow$ multi-expert strategy voting $\Rightarrow$ strategy execution. Each expert agent prioritises current subgoals, and the system ranks experts based on average priority, determines the decision-making order, reaches consensus, executes the most supported strategy, and implements a reward mechanism.

Compared to the original collaboration model, the new agent collaboration model not only integrates SledgeHammer's automated theoretical proof capabilities but also features a more intelligent and robust strategy selection evaluation mechanism. The involvement of multiple experts significantly enhances the reliability of decision-making, while concurrent multi-session operations further improve efficiency. Each agent can be specialised for specific tasks, and more agents can provide stronger proof capabilities. Standardised interfaces not only make the system easier to expand and maintain but also reduce integration costs through modularisation. The multi-expert collaboration model described above is just one proposed implementation method; developers are encouraged to experiment with various architectures based on the agent platform.

4.2.2 Learning Costs

This project has a large amount of code, which is difficult for new developers and requires a significant learning investment. To optimise the user experience, this project has reasonably named and annotated each implemented method and class, and provided detailed documentation for developers to learn and familiarise themselves with the project. According to the instructions, the entire process includes environment configuration, dependency installation, and service startup, with a total time of approximately 15 minutes, indicating that the project deployment process is reasonable and smooth.

4.3 Robustness

The system has added a large amount of visualisation code during implementation and operation, exposing session information and interactions in the gym environment to the front-end interface as much as possible. In most scenarios, the system uses robust exception handling mechanisms instead of direct crashes and leverages Scala middleware to transmit server-side error information to the client, ensuring the stability of the agent during the proof search process. Additionally, through concurrency testing, the backend server can stably handle multiple concurrent client requests without exceeding the memory threshold.

4.4 Comparison

4.4.1 Performance

When executing commands across multiple sessions, the delay in session startup is greatly eliminated for frequently used configurations. The system performed timing statistics for session creation under two settings: with caching enabled and with caching disabled.

Through five rounds of concurrent testing with six sessions, the average data results for each round were statistically analysed and recorded. The experimental results clearly show that enabling the cache capability improves the efficiency of creating sessions with the same theoretical header by nearly half.

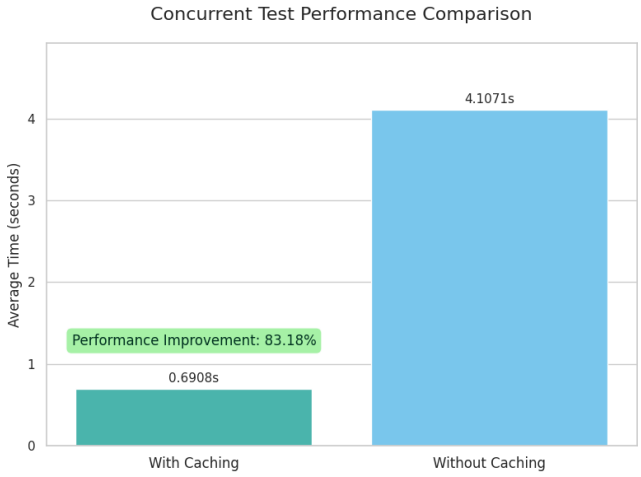

Figure 4.3: Performance Comparison

4.4.2 Comparison with projects in other ITPs

Comparison with the original IsabelleGym: Compared with its predecessor, this project has achieved improvements in many aspects. The current IsabelleGym truly achieves compatibility with the latest version, rather than just staying at the Isabelle version from a few years ago. Malicious bugs such as server startup, vectorised environment fixes, and inconsistent unit test formats have been fixed, and a large amount of exception handling work has been added to maintain the original code. The interactive interface has also been optimised, meaningless data caching in the backend has been removed, and more comprehensive exception handling has been provided. This work is a prerequisite for all subsequent front-end and back-end work and is also crucial for the iteration of the entire project.

Comparison with Kimina-Prover and Pantograph: The Lean community is still leading in the integration of machine learning and proof assistants, especially in the integration of large language models and advanced reasoning capabilities. By leveraging their excellent design methods, this project has successfully implemented a higher-level foundation platform for the Isabelle community, aligning it in terms of architectural robustness, interface generality, and developer friendliness. It also leverages the characteristics of Isabelle and the functional foundation of IsabelleGym to provide many capabilities that the Lean community does not have, including the choice between declarative and procedural styles for strategies, multi-server, multi-session, and multi-theory environment pipelines, among other project-specific capabilities.

Chapter 5

Conclusion

This project successfully maintained and optimised the original IsabelleGym environment, expanded its front-end and back-end capabilities, evaluated it against other excellent implementations in the field of automated theorem proving, and provided a more promising and general-purpose Isabelle proof environment.

5.1 Contributions

1. **Complete Agent Interface Construction** This project completed agent-based interactive proof in a variety of example problem scenarios, provided a unified standardised interface, and successfully utilised the MCTS search algorithm to demonstrate the complete system functionality, proving the successful integration of machine learning agents and the Isabelle environment, and reserving an interface for the subsequent integration of large language model automation capabilities.

2. **Backend LRU Cache System and System Integration** Through the implementation of this section, the backend of IsabelleGym has introduced LRU eviction mechanisms, cache key design, multi-session parallelism, load balancing, and memory management capabilities, providing efficient infrastructure for agent applications in high-concurrency scenarios.

3. **Simplified deployment solutions for various types of projects** This project provides a one-click quick deployment method and adds sufficient comments and class method descriptions to various functional codes, greatly simplifying the project's learning curve and reducing the learning cost.

5.2 Lessons Learnt

The IsabelleGym project has a huge code base, complex interaction logic, and numerous features, which posed significant challenges for me in the early stages of getting started and familiarising myself with the project. This made me realise the importance of technical documentation for developers.

I had little prior knowledge of mathematical theorem-proving assistants and had never encountered similar research before. This led me to frequently consult the official Isabelle documentation and open-source project implementations while working on interaction design, which also gave me new insights into the fields of interaction theorem proving and machine learning.

When formalising problem proofs and integrating machine learning algorithms, I always got too caught up in the design of specific example scenarios, the writing of problem formalisation theory documents, and the repair of various exceptions. This was a clear mistake, as these attempts consumed a lot of my energy and time. In fact, I didn't need to spend too much time on creating specific problem environments and training models, because the original IsabelleGym already provided these capabilities, and my work would not affect the original functionality.

When I first started building the agent interface, I focused on how to use the functions exposed by the gym environment and ignored the essential interaction logic. I should have chosen to understand the system architecture from the bottom up as quickly as possible, then deeply analysed the underlying code to understand the interaction principles of the project, which would have allowed me to develop and optimise the front end more efficiently.

In addition, I should have used an MVP-style iterative delivery method from the beginning, managing versions and committing to git after each small executable feature was implemented, as well as recording and retaining development logs, which would have been extremely helpful for the entire project. In such a tight and short project timeframe, it was necessary to first clarify the core requirements of the project, start from reality, and formulate a more reasonable development plan, rather than setting unachievable goals.

5.3 Limitations

Although this research has been completed in many aspects and is comparable to the latest excellent projects in the field, there are still many limitations and issues.

1. Limitations of computing resources: Due to memory and computing power limitations, I was unable to conduct large-scale testing, more complex reinforcement learning training, and truly integrate large model capabilities.

2. Technical limitations: Due to the incompleteness of the IsabelleGym base project, which is not a true production environment, even though I have fixed many bugs, it is still possible that some issues have not been completely resolved. Currently, I can only verify through simple example problems and test scripts, and have not demonstrated the system in real-world scenarios with high concurrency.

3. Project Time Constraints: The time allocated for this project is limited, and the learning curve and difficulty of implementation are significant. This makes it impractical for me to undertake further architectural restructuring or performance optimisation on such a large-scale project.

5.4 Future Work

In future, the follow-up work of this project can be divided into several aspects:

Further optimisation of system performance. Selection of communication technologies more suitable for distributed training, reduction of time costs for interaction with the Isabelle environment, and data buffering.

Integrating large language model capabilities into IsabelleGym. LLM-based theorem proving requires higher quality and more structured training data, optimisation of data extraction granularity, data storage structure and format. Other tasks include constructing a fine-tuned data set for Isabelle that can utilise enhanced data extraction and serialisation capabilities to process large-scale proof libraries such as AFP, and training LLM optimised for Isabelle syntax and proof style.

The lifelong learning concept can be referenced. Intelligent agents continuously attempt to prove new theorems and add successful results to the known database and

training data, achieving continuous growth in capabilities through an ever-expanding knowledge domain [10].

In summary, this research work is an optimisation and extension of the original IsabelleGym, drawing on and experimenting with many advanced works, and providing a bridge for further reinforcement learning and LLM training. I hope that future research will truly enable Isabelle to catch up with community projects such as Lean and continue to move forward towards the AI mathematicians that mathematical researchers are looking forward to.

Bibliography

[1] Leni Aniva, Chuyue Sun, Brando Miranda, Clark Barrett, and Sanmi Koyejo. Pantograph: A machine-to-machine interaction interface for advanced theorem proving, high level reasoning, and data extraction in lean 4, 2025.

[2] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. *The Tactician: A Seamless, Interactive Tactic Learner and Prover for Coq*, page 271277. Springer International Publishing, 2020.

[3] Jasmin Christian Blanchette, Sascha Böhme, and Lawrence C. Paulson. Extending sledgehammer with smt solvers. In Nikolaj Bjørner and Viorica Sofronie-Stokkermans, editors, *Automated Deduction – CADE-23*, pages 116–130, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[4] Davide Castelvecchi. Deepmind hits milestone in solving maths problems ais next grand challenge. *Nature (London)*, 632(8024):236–237, 2024.

[5] Maxwell Crouse, Ibrahim Abdelaziz, Bassem Makni, Spencer Whitehead, Cristina Cornelio, Pavan Kapanipathi, Kavitha Srinivas, Veronika Thost, Michael Witbrock, and Achille Fokoue. A deep reinforcement learning approach to first-order logic theorem proving, 2020.

[6] Fabian Huch and Makarius Wenzel. Distributed Parallel Build for the Isabelle Archive of Formal Proofs. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 22:1–22:19, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[7] Steven James, George Konidaris, and Benjamin Rosman. An analysis of monte carlo tree search. *Proceedings of the AAAI Conference on Artificial Intelligence*, 31(1), Feb. 2017.

[8] Albert Q. Jiang, Wenda Li, J. M. Han, and Yong Wu. LISA: Language models of ISAbelle proofs. In *6th Conference on Artificial Intelligence and Theorem Proving, AITP 2021*, pages 378–392, 2021.

[9] Levente Kocsis and Csaba Szepesvari. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, 2006.

[10] Adarsh Kumarappan, Mo Tiwari, Peiyang Song, Robert Joseph George, Chaowei Xiao, and Anima Anandkumar. Leanagent: Lifelong learning for formal theorem proving, 2025.

[11] N. Megiddo and D.S. Modha. Outperforming lru with an adaptive replacement cache algorithm. *Computer*, 37(4):58–65, 2004.

[12] Tobias. Nipkow, Lawrence C. Paulson, and Markus. Wenzel. *Isabelle/HOL : A Proof Assistant for Higher-Order Logic*. Lecture Notes in Computer Science, 2283. Springer Berlin Heidelberg, Berlin, Heidelberg, 1st ed. 2002. edition, 2002.

[13] Lawrence C. Paulson. The foundation of a generic theorem prover, 2000.

[14] Lawrence Charles Paulson and Tobias Nipkow. Isabelle: A generic theorem prover. 1994.

[15] Avi Press, Michael Pacer, Thomas Griffiths, and Brian Christian. Caching algorithms and rational models of memory. *Proceedings of the Annual Meeting of the Cognitive Science Society*, 36(36), 2014.

[16] Johannes Scheiermann and Wolfgang Konen. Alphazero-inspired game learning: Faster training by using mcts only at test time. *IEEE Transactions on Games*, 15(4):637647, December 2023.

[17] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy P. Lillicrap, Fan Hui, L. Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017.

[18] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean copilot: Large language models as copilots for theorem proving in lean, 2025.

[19] Terence Tao. Machine-assisted proof. *Notices of the American Mathematical Society*, 72(1):6–13, 2025.

[20] The Stanford Centaur Lab. PyPantograph. https://github.com/stanford-centaur/PyPantograph, 2020.

[21] Milan Tom. Isabellegym: A learning environment for reinforcement learning with isabelle/hol. Bachelor's thesis, University of Cambridge, 2025. Unpublished.

[22] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulo, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierr, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024.

[23] Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxc, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, Jiawei Liu, Jonas Bayer, Julien Michel, Longhui Yu, Lo Dreyfus-Schmidt, Lewis Tunstall, Luigi Pagani, Moreira Machado, Pauline Bourigault, Ran Wang, Stanislas Polu, Thibaut Barroyer, Wen-Ding Li, Yazhe Niu, Yann Fleureau, Yangyang Hu, Zhouliang Yu, Zihan Wang, Zhilin Yang, Zhengying Liu, and Jia Li. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning, 2025.

[24] Makarius Wenzel. Asynchronous user interaction and tool integration in isabelle/pide. In Gerwin Klein and Ruben Gamboa, editors, *Interactive Theorem Proving*, pages 515–530, Cham, 2014. Springer International Publishing.

[25] Makarius Wenzel. Isabelle/jedit as ide for domain-specific formal languages and informal text documents. *Electronic Proceedings in Theoretical Computer Science*, 284:7184, November 2018.

[26] Makarius Wenzel. The isabelle system manual, 2025.

[27] Markus Wenzel. Isar — a generic interpretative approach to readable formal proof documents. In Yves Bertot, Gilles Dowek, Laurent Théry, André Hirschowitz, and Christine Paulin, editors, *Theorem Proving in Higher Order Logics*, pages 167–183, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.

[28] Markus Wenzel and Tu Munchen. The isabelle/isar reference manual. 03 2001.

[29] Kaiyu Yang and Jia Deng. Learning to prove theorems via interacting with proof assistants, 2019.

[30] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models, 2023.

[31] Maciej wiechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Madziuk. Monte carlo tree search: a review of recent modifications and applications. *Artificial Intelligence Review*, 56(3):24972562, July 2022.

Appendix A

Data Extraction

A.1 Data Format

Integrating with data extraction capabilities, each step of the complete proof status can be extracted and retained, facilitating supervised learning.

JSON Schema for Proof Data

Root Object

- **type**: ‘object‘
- **properties**:
  - **input**:
    - **type**: ‘object‘
    - **properties**:
      - **goal**: ‘string‘ (Description: The current proof goal)
      - **all_subgoals**: ‘array‘ (Items: ‘string‘)
      - **goal_id**: ‘integer‘ (Minimum: 0)
      - **theorem**: ‘string‘ (Description: The theorem statement)
      - **context**: ‘array‘ (Items: ‘string‘)
      - **proof_step**: ‘integer‘ (Minimum: 0)
      - **depth**: ‘integer‘ (Minimum: 0)
    - **required**: ‘[”goal”, ”goal_id”, ”theorem”]‘
  - **target**: ‘string‘ (Description: The executed tactic)

- **success**: ‘boolean‘
- **metadata**:
  - **type**: ‘object‘
  - **properties**:
    - **execution_time**: ‘number‘ (Minimum: 0)
    - **subgoals_change**: ‘string‘ (Pattern: ‘integer to integer‘)
    - **agent**: ‘string‘
    - **timestamp**: ‘string‘ (Format: ‘date-time‘)

- **required**: ‘[”input”, ”target”, ”success”]‘